Sistemi Operativi e Reti di Calcolatori (SOReCa)

Corso di Laurea in *Ingegneria Informatica e Automatica (BIAR)*
Terzo Anno | Primo Semestre
A.A. 2025/2026

Esercitazione [09] Semplice HTTP server “from scratch”

Riccardo Lazzeretti lazzeretti@diag.uniroma1.it
Paolo Ottolino paolo.ottolino@uniroma1.it
Matteo Cornacchia cornacchia@diag.uniroma1.it

DIPARTIMENTO DI INGEGNERIA INFORMATICA AUTOMATICA E GESTIONALE ANTONIO RUBERTI

SAPIENZA UNIVERSITÀ DI ROMA

Sommario

- Soluzioni precedente esercitazione

- Simple HTTP Server:
  - Caratteristiche
  - Recap su Socket TCP/IP: `socket()`,`bind()`,`listen()`,`accept()`
  - RFC HTTP/1: RFC 1945

- Lab09, es. 1: nano HTTP server
  - Ascolto: Elaborare Thread-based (Pooling), GET valida, nome file e sua codifica
  - Risposta: tipo MIME, Ricerca file e sua Esistenza, Messaggio HTTP

Dipartimento di Ingegneria informatica
automatica e gestionale Antonio Ruberti

Sapienza
Università di Roma

Soluzione: Multi-Process PoC

Lab08, es1 EchoServer

Multi-Process: EchoServer

Lab08, es1

Completare il codice dell'EchoServer in modalità multi-process

Sorgenti

- `Makefile`
- Client: `client.c`
- Server: `server.c`
- compilazione: `-DSERVER_MPROC` vs `-DSERVER_SERIAL`

Suggerimento: seguire i blocchi di commenti inseriti nel codice

Altro suggerimento:

Per monitorare a runtime il numero di istanze di processi attivi in un certo momento, lanciare da terminale il comando:

```
ps -e -O ppid | head -1; ps -e -O ppid | grep multiprocess
```

Dipartimento di Ingegneria informatica automatica e gestionale Antonio Ruberti

Sapienza Università di Roma

Multi-Process: EchoServer

Lab08, es1 es2 - client

```
 int main(int argc, char* argv[]) {
    int ret,bytes_sent,recv_bytes;
```

```
// display welcome message from server    recv_bytes = 0;
    recv_bytes = 0;
    do {
        ret = recv(socket_desc, buf + recv_bytes, buf_len - recv_bytes, 0);
        if (ret == -1 && errno == EINTR) continue; // system call interrotta da
// segnale → ignora errore e continua.
        if (ret == -1) handle_error("Cannot read from the socket"); //errore di
// lettura reale → terminazione del programma.
        if (ret == 0) break; // l'altro peer ha chiuso la comunicazione → uscita dal
// ciclo.
    recv_bytes += ret;
    } while ( buf[recv_bytes-1] != '\n' );
    printf("%s", buf);
```

```
// send message to server
    bytes_sent = 0;
    while ( bytes_sent < msg_len) {
        ret = send(socket_desc, buf + bytes_sent, msg_len - bytes_sent, 0);
        if (ret == -1 && errno == EINTR) continue; // system call interrotta da
// segnale → ignora errore e continua.
        if (ret == -1) handle_error("Cannot read from the socket"); //errore di
// lettura reale → terminazione del programma.
        bytes_sent += ret;
    }
```

```
 /** TODO: set up parameters for the connection and initiate it */
    server_addr.sin_addr.s_addr = inet_addr(SERVER_ADDRESS);
    server_addr.sin_family      = AF_INET;
    server_addr.sin_port        = htons(SERVER_PORT); // net byte order!
    // initiate a connection on the socket
    ret = connect(socket_desc, (struct sockaddr*) &server_addr, sizeof(struct
sockaddr_in));
    if (ret < 0) handle_error("Could not create connection");
```

```
      // read message from server
         * Suggestions:
         * - recv() with flags = 0 is equivalent to read() from a descriptor
         * - for sockets, we get a 0 return value only when the peer closes
         *   the connection: if there are no bytes to read and we invoke
         *   recv() we will get stuck, because the call is blocking!
         * - deal with partially sent messages (we do not know the message size) */

    recv_bytes = 0;
        do {
            ret = recv(socket_desc, buf + recv_bytes, buf_len - recv_bytes, 0);
            if (ret == -1 && errno == EINTR) continue; // system call interrotta da segnale →
// ignora errore e continua.
        if (ret == -1) handle_error("Cannot read from the socket"); //errore di lettura reale →
// terminazione del programma.
        if (ret == 0) break; // l'altro peer ha chiuso la comunicazione → uscita dal ciclo.
            recv_bytes += ret;
        } while ( buf[recv_bytes-1] != '\n' );
```

DIPARTIMENTO DI INGEGNERIA INFORMATICA
AUTOMATICA E GESTIONALE ANTONIO RUBERTI

SAPIENZA
UNIVERSITÀ DI ROMA

Multi-Process: EchoServer

Lab08, es1 es2 – server (`$ make multiprocess || multithread`)

```
// parse client IP address and port
    char client_ip[INET_ADDRSTRLEN];
    inet_ntop(AF_INET, &(client_addr->sin_addr), client_ip,
INET_ADDRSTRLEN);
    uint16_t client_port = ntohs(client_addr->sin_port); // port number is
an unsigned short
```

```
// send welcome message
    sprintf(buf, "Hi! I'm an echo server. You are %s talking on port
%hu.\nI will send you back whatever"
            " you send me. I will stop if you send me %s :-)\n",
client_ip, client_port, quit_command);
    msg_len = strlen(buf);
    int bytes_sent = 0;
    while ( bytes_sent < msg_len) {
        ret = send(socket_desc, buf + bytes_sent, msg_len - bytes_sent,
0);
        if (ret == -1 && errno == EINTR) continue; // system call
interrotta da
// segnale → ignora errore e continua.
        if (ret == -1) handle_error("Cannot read from the socket");
//errore di
// lettura reale → terminazione del programma.
            bytes_sent += ret;
    }
```

```
   // echo loop
    while (1) { // read message from client
        memset(buf, 0, buf_len);
        recv_bytes = 0;
        do {
            ret = recv(socket_desc, buf + recv_bytes, 1, 0);
            if (ret == -1 && errno == EINTR) continue; // system call interrotta da
// segnale → ignora errore e continua.
        if (ret == -1) handle_error("Cannot read from the socket"); //errore di
// lettura reale → terminazione del programma.
        if (ret == 0) break; // l'altro peer ha chiuso la comunicazione → uscita dal
// ciclo.
        } while ( buf[recv_bytes++] != '\n' );
```

```
        // check whether I have just been told to quit...
        if (recv_bytes == 0) break;
        if (recv_bytes == quit_command_len && !memcmp(buf, quit_command,
quit_command_len)) break;
```

```
    // ... or if I have to send the message back
        bytes_sent=0;
        while ( bytes_sent < recv_bytes) {
            ret = send(socket_desc, buf + bytes_sent, recv_bytes - bytes_sent, 0);
            if (ret == -1 && errno == EINTR) continue; // system call interrotta da
// segnale → ignora errore e continua.
        if (ret == -1) handle_error("Cannot read from the socket"); //errore di
// lettura reale → terminazione del programma.
            bytes_sent += ret;
        }
    }
```

DIPARTIMENTO DI INGEGNERIA INFORMATICA
AUTOMATICA E GESTIONALE ANTONIO RUBERTI

SAPIENZA
UNIVERSITÀ DI ROMA

Multi-Process: EchoServer

Lab08, es1 – server (`$ make multiprocess`)

```
#elif SERVER_MPROC


void mprocServer(int server_desc) {
    int ret = 0;
    // we initialize client_addr to zero
    struct sockaddr_in client_addr = {0};
    // loop to manage incoming connections forking the server process
    int sockaddr_len = sizeof(struct sockaddr_in);
    while (1) {
        // accept incoming connection
        int client_desc = accept(server_desc, (struct sockaddr *) &client_addr,
(socklen_t *)&sockaddr_len);
        // check for interruption by signals
        if (client_desc == -1 && errno == EINTR) continue;
        if (client_desc < 0) handle_error("Cannot open socket for incoming
connection");
        if (DEBUG) fprintf(stderr, "Incoming connection accepted...\n");
        pid_t pid = fork();
        if (pid < 0) handle_error("Cannot fork server process to handle the request");
        else if (pid == 0) {
            // child: close the listening socket and process the request
            ret = close(server_desc);
            if (ret) handle_error("Cannot close listening socket in child process");
            connection_handler(client_desc, &client_addr);
            if (DEBUG) fprintf(stderr, "Process created to handle the request has
completed.\n");
            _exit(EXIT_SUCCESS);
        }
        else {
            // server: close the incoming socket and continue
            ret = close(client_desc);
            if (ret) handle_error("Cannot close incoming socket in main process");
            if (DEBUG) fprintf(stderr, "Child process created to handle the
request!\n");
            // reset fields in client_addr so it can be reused for the next accept()
            memset(&client_addr, 0, sizeof(struct sockaddr_in));
        }
    }
}
```

```
 #ifdef SERVER_SERIAL
// nothing to do here
#elif SERVER_MPROC
// nothing to do here
#elif SERVER_MTHREAD
#include <pthread.h>
typedef struct handler_args_s
{   int socket_desc;
    struct sockaddr_in* client_addr;
} handler_args_t;
#endif
```

DIPARTIMENTO DI INGEGNERIA INFORMATICA
AUTOMATICA E GESTIONALE ANTONIO RUBERTI

SAPIENZA
UNIVERSITÀ DI ROMA

Soluzione Multi-Thread PoC

Lab08, es2 EchoServer «Event-driven»

Multi-Thread: EchoServer

Lab08, es2

Completare il codice dell'EchoServer in modalità multi-thread

Sorgenti

- `Makefile`
- Client: `client.c`
- Server: `server.c`
- compilazione: `-DSERVER_MTHREAD` vs `-DSERVER_SERIAL`

Suggerimento: seguire i blocchi di commenti inseriti nel codice

Altro suggerimento:

Per monitorare a runtime il numero di istanze di processi/thread attivi in un certo momento, lanciare da terminale il comando:

```
ps -e -T | head -1 ; ps -e -T | grep multithread
```

Dipartimento di Ingegneria informatica automatica e gestionale Antonio Ruberti

Sapienza Università di Roma

Multi-Process: EchoServer

Lab08, es2 – server (`$ make multithread`)

```
void mthreadServer(int server_desc) {
    int ret = 0;
    // loop to manage incoming connections spawning handler threads
    int sockaddr_len = sizeof(struct sockaddr_in);
    while (1) {
        // we dynamically allocate a fresh client_addr object at each
        // loop iteration and we initialize it to zero
        struct sockaddr_in* client_addr = calloc(1, sizeof(struct sockaddr_in));

        // accept incoming connection
        int client_desc = accept(server_desc, (struct sockaddr *) client_addr,
(socklen_t *)&sockaddr_len);

        if (client_desc == -1 && errno == EINTR) continue; // check for
interruption by signals
        if (client_desc < 0) handle_error("Cannot open socket for incoming
connection");

        if (DEBUG) fprintf(stderr, "Incoming connection accepted...\n");
        pthread_t thread;

        // prepare arguments for the new thread
        handler_args_t *thread_args = malloc(sizeof(handler_args_t));
        thread_args->socket_desc = client_desc;
        thread_args->client_addr = client_addr;
        ret = pthread_create(&thread, NULL, thread_connection_handler, (void
*)thread_args);
        if (ret) handle_error_en(ret, "Could not create a new thread");

        if (DEBUG) fprintf(stderr, "New thread created to handle the
request!\n");

        ret = pthread_detach(thread); // I won't phtread_join() on this thread
        if (ret) handle_error_en(ret, "Could not detach the thread");
    }
}
```

```
 #ifdef SERVER_SERIAL
// nothing to do here
#elif SERVER_MPROC
// nothing to do here
#elif SERVER_MTHREAD
#include <pthread.h>
typedef struct handler_args_s
{   int socket_desc;
    struct sockaddr_in* client_addr;
} handler_args_t;
#endif
```

```
#elif SERVER_MTHREAD


// Wrapper function that take as input handler_args_t struct and then call
// connection_handler.
void *thread_connection_handler(void *arg) {
    handler_args_t *args = (handler_args_t *)arg;
    int socket_desc = args->socket_desc;
    struct sockaddr_in *client_addr = args->client_addr;
    connection_handler(socket_desc,client_addr);
    // do not forget to free client_addr! by design it belongs to the
    // thread spawned for handling the incoming connection
    free(args->client_addr);
    free(args);
    pthread_exit(NULL);
}
```

DIPARTIMENTO DI INGEGNERIA INFORMATICA
AUTOMATICA E GESTIONALE ANTONIO RUBERTI

SAPIENZA
UNIVERSITÀ DI ROMA

Simple HTTP server

Caratteristiche
Recap su Socket TCP/IP: `socket()`,`bind()`,`listen()`,`accept()`
RFC HTTP/1: RFC 1945

Simple HTTP Server

Caratteristiche

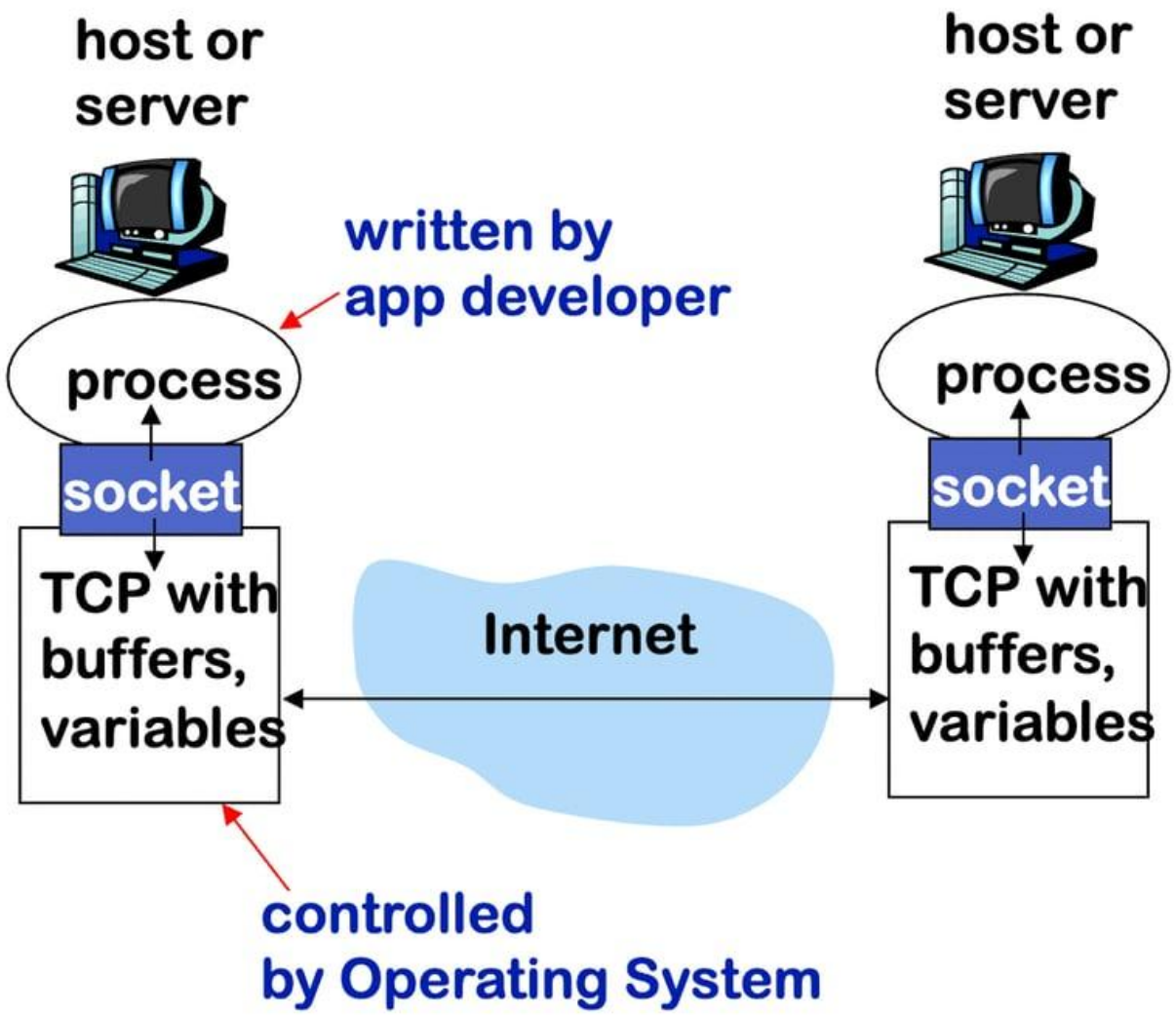
Il server HTTP deve effettuare le seguenti operazioni:

1. ascoltare le connessioni in arrivo su una **porta TCP 8080**. → `socket()`, `bind()`, `listen()`
2. elaborare le richieste dei client **in thread separati** (più connessioni simultanee. → `pthread_create()`
3. accettare solo **richieste** dal client valide di tipo **GET**. → gestione GET non valida
4. estrarre il **nome file** richiesto (eliminare «.», evitare directory traversal)
5. decodificare il **nome** file **codificato** in URL (ATTENZIONE: i caratteri speciali sono codificati; es. « » diventa %20)
6. ricercare il file **senza distinzione** tra **maiuscole** e **minuscole** (ATTENZIONE: in Unix «A» e «a» sono caratteri distinti).
7. verificare se il **file** richiesto **esiste** nella directory corrente. → gestione «**404 Not Found**».
8. determinare l **estensione** del **file** per identificare il **tipo MIME** appropriato per la risposta
9. creare una **risposta HTTP** con le intestazioni appropriate, incluso il tipo MIME determinato, e invia il contenuto del file al client.

Il server è un demone: continua ad accettare ed elaborare le connessioni in arrivo finché non viene terminato manualmente (Ctrl-C, kill -9 etc.).

Simple HTTP Server

1. Ascoltare le connessioni in arrivo – TCP Sockets in Unix 1/2



Se un processo in esecuzione su un host è come una casa, un socket è come una porta che lascia entrare e uscire la posta.

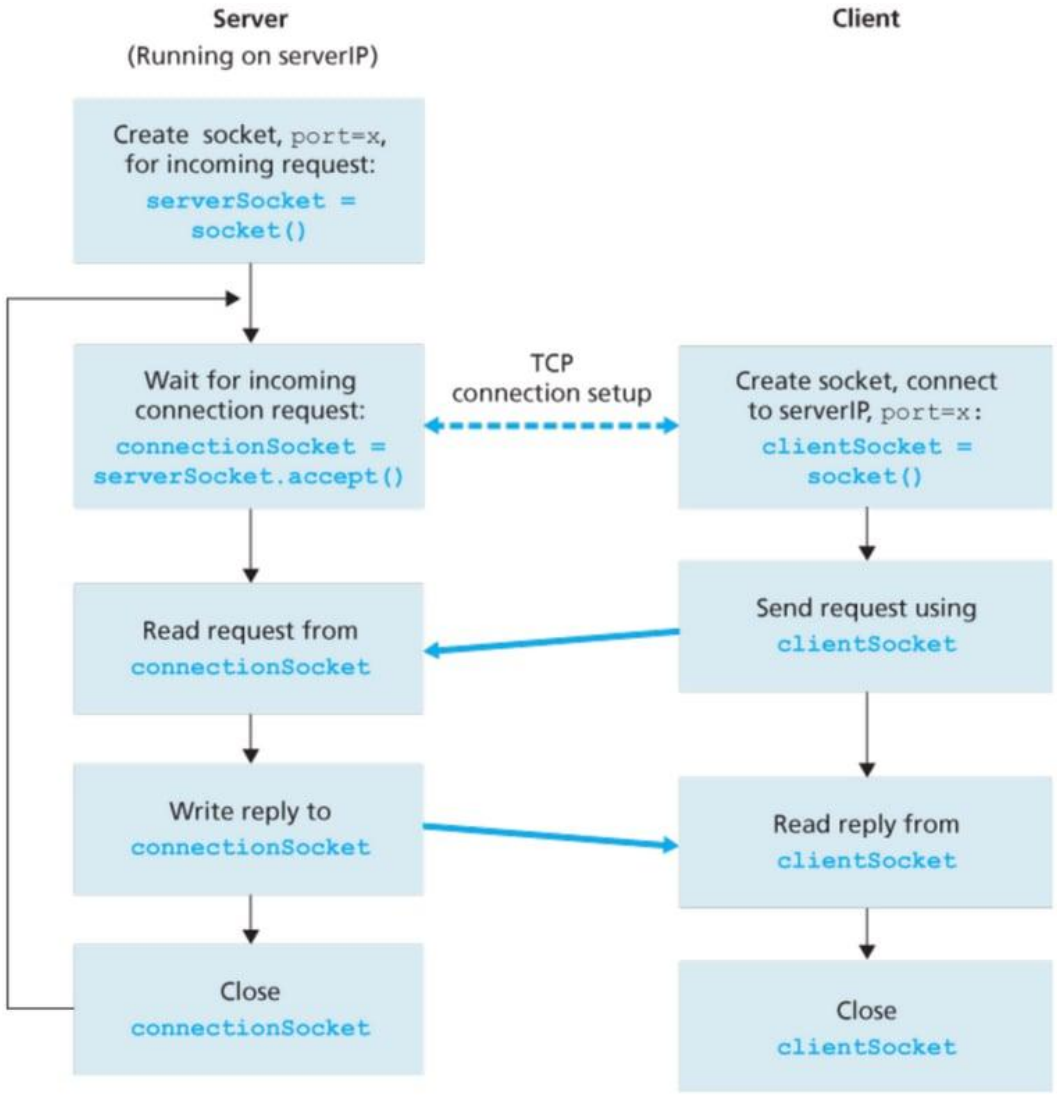
Quando una persona in casa riceve o consegna una lettera, può essere agnostica su come la posta viene consegnata nel mondo esterno.

Dipartimento di Ingegneria informatica automatica e gestionale Antonio Ruberti

Sapienza Università di Roma

Simple HTTP Server

1. Ascoltare le connessioni in arrivo – TCP Sockets in Unix 2/2



Utilizzando l interfaccia di rete socket, è possibile redigere il codice del server web traendo vantaggio dal set di funzioni dal pacchetto standard C `<sys/socket.h>` che consentono al server di «dialogare con" i client tramite Internet.
Anche i client utilizzano socket per parlare con il server web.
Il socket è il meccanismo consensuale con cui comunicare tramite Internet.

Dipartimento di Ingegneria informatica automatica e gestionale Antonio Ruberti

Sapienza Università di Roma

Simple HTTP Server

1. Ascoltare le connessioni in arrivo – UDP, TCP Sockets in C

1. creare una socket per esporre le funzionalità del server: `socket()`:
   1. configurare la famiglia di socket richiesta (`sin_family = AF_INET`)
   2. configurare l indirizzo IPv4 (`sin_addr.s_addr = INADDR_ANY`)
   3. configurare il numero di porta (`sin_port = htons(PORT)`).

Nel server HTTP, la configurazione del socket segue l'URL `http://localhost:8080` usando il numero di porta `8080`.

2. associare il socket alla porta (`bind(server_fd, (struct sockaddr *)&server_addr, sizeof(server_addr))`)
   - una volta associato, **altre applicazioni o processi** utente **non** potranno **acquisire quel socket** fino al rilascio da parte del processo server
   - Ovvero, dal momento del **`bind()`**, il processo server avrà il **pieno controllo** su quella **porta**.
   - Finché il **server** sarà **in esecuzione**, la **porta** sarà **riservata** a quel server.
   - Questo è il **meccanismo** con cui il **kernel** (che gestisce tutte le comunicazioni TCP/IP) **associa univocamente** tutte le **connessioni** su una determinata **porta** ad un determinato **processo**.
   - Anche **qualora non** sia in **ascolto** su **tutte le interfacce** di rete (cioè su tutti gli IP) dell'host, la **porta** è **riservata** ad esso **dappertutto**, anche dove non in ascolto.
   - Dopo il **`bind()`** avvenuto con **successo**, siamo sicuri che **tutte** le **comunicazioni** su quella **porta** siano veicolate verso il **server**, potendo ascoltare da quella porta le richieste utentem senza perderne alcuna.

Simple HTTP Server

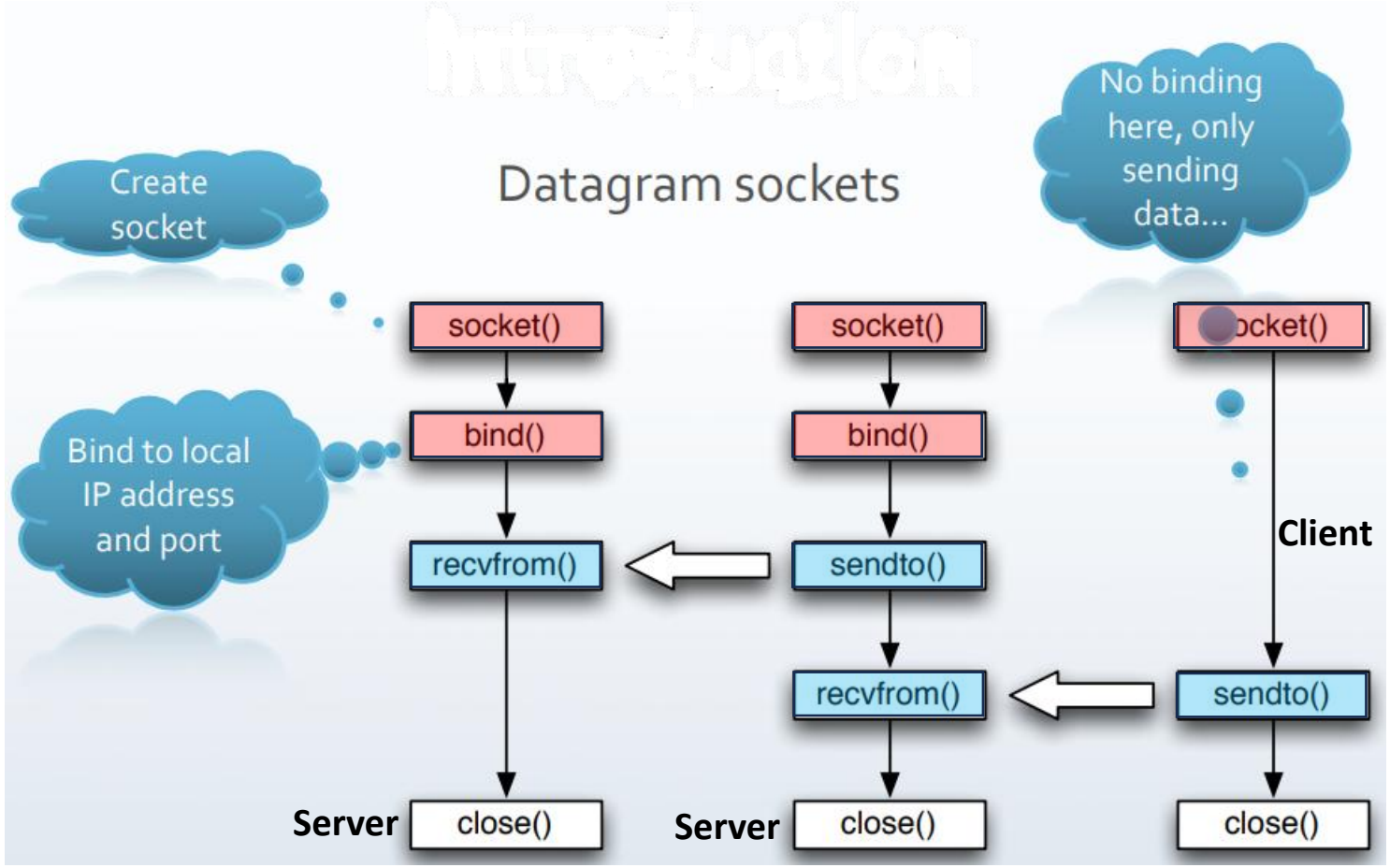
1. Ascoltare le connessioni in arrivo – UDP, TCP Sockets in C

3. Mettersi in ascolto sul socket (**solo TCP**): `listen()`
4. Accettare connessioni sulla socket (**solo TCP**): `accept()`
5. Ricevere ed inviare i dati sulla socket, per ogni connessione: `send()` `recv()`:
   - **UDP**: `recvfrom()`, `sendto()`
   - **TCP**: `recv()`,`send()`

6. Chiudere il socket del client (**solo TCP**): `close(client_sock)`
7. Chiudere il socket del client: `close(server_fd)`

Simple HTTP Server

How Socket Works – DataGram 1/2



- No connection and unreliable the same socket can be used to send/receive
- Datagrams to/from multiple processes
- uniquely identified by the IP addresses and port numbers (remote and local) for:
  - client (`socket()`)
  - server (`socket()`, `bind()`)
- No syntax distinction between server and client, just:
  - `sendto()`
  - `recvfrom()`

Dipartimento di Ingegneria informatica automatica e gestionale Antonio Ruberti

Sapienza Università di Roma

Simple HTTP Server

Ascoltare le connessioni in arrivo - UDP Sockets in C

```
#include <stdio.h>
#include <string.h>
#include <sys/socket.h>
#include <arpa/inet.h>

int main(void){
    int socket_desc;
    struct sockaddr_in server_addr, client_addr;
    char server_message[2000], client_message[2000];
    int client_struct_length = sizeof(client_addr);

    // Clean buffers:
    memset(server_message,  \0 , sizeof(server_message));
    memset(client_message,  \0 , sizeof(client_message));

    // Create UDP socket:
    socket_desc = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);
        if(socket_desc < 0){/* AF_NET: use IPv4 (no IPv6); SOCK_DGRAM: UDP */
        printf("Error while creating socket\n");
        return -1;    }
    printf("Socket created successfully\n");

    // Set port and IP:
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(2000);
    server_addr.sin_addr.s_addr = inet_addr("127.0.0.1");

    // Bind to the set port and IP:
    if(bind(socket_desc, (struct sockaddr*)&server_addr, sizeof(server_addr))
< 0){
        printf("Couldn t bind to the port\n");
        return -1;    }
    printf("Done with binding\n");

    printf("Listening for incoming messages...\n\n");
```

```
// Receive clients message:
if (recvfrom(socket_desc, client_message, sizeof(client_message), 0,
     (struct sockaddr*)&client_addr, &client_struct_length) < 0){
    printf("Couldn t receive\n");
    return -1;    }
printf("Received message from IP: %s and port: %i\n",
       inet_ntoa(client_addr.sin_addr), ntohs(client_addr.sin_port));

printf("Msg from client: %s\n", client_message);

// Respond to client:
strcpy(server_message, client_message);
if (sendto(socket_desc, server_message, strlen(server_message), 0,
     (struct sockaddr*)&client_addr, client_struct_length) < 0){
    printf("Can t send\n");
    return -1;    }

// Close the socket: un solo messaggio e poi muore
close(socket_desc);
return 0;
}
```

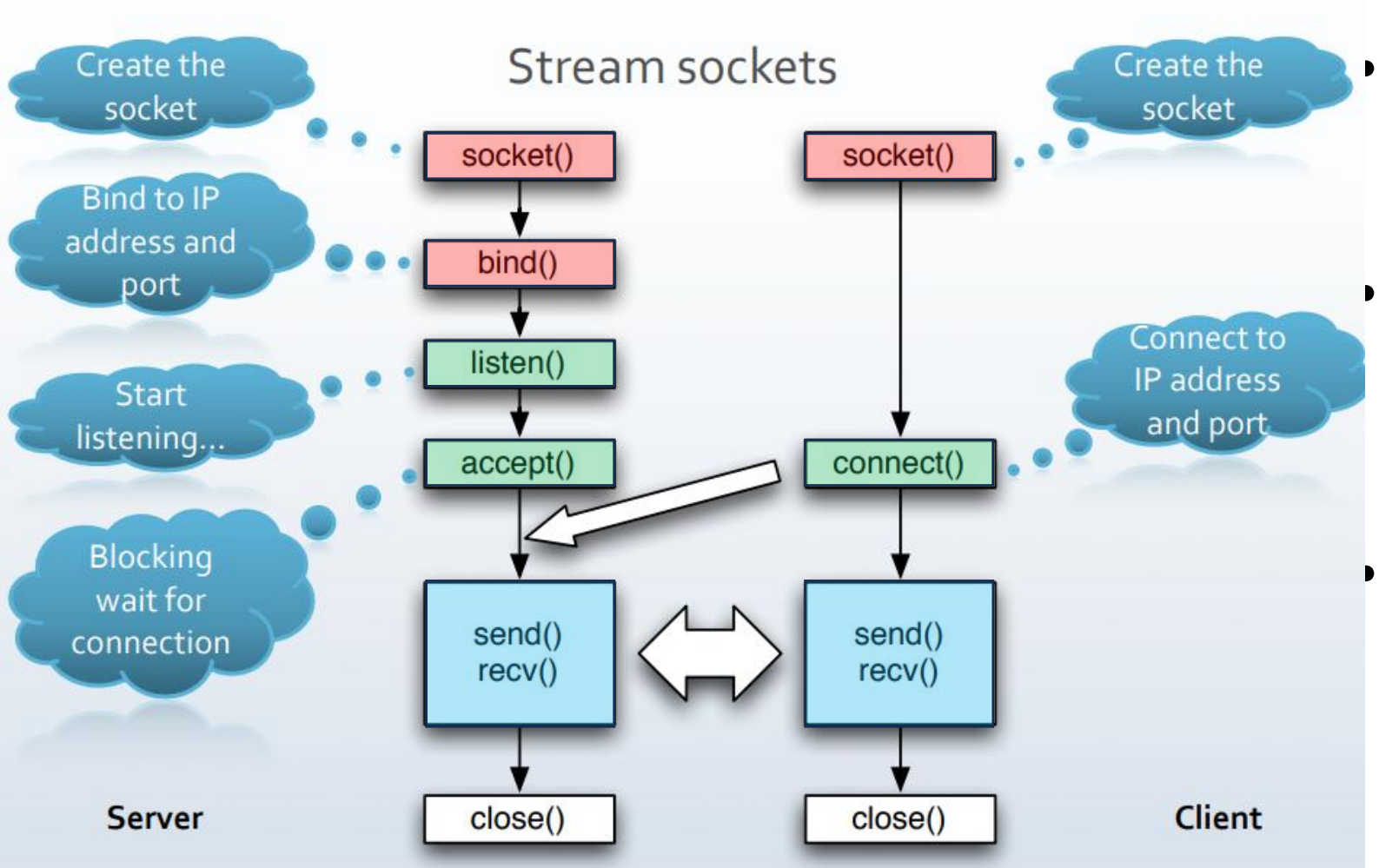
Ma il protocollo HTTP prevede una connessione TCP, quindi un socket TCP, non UDP…

DIPARTIMENTO DI INGEGNERIA INFORMATICA AUTOMATICA E GESTIONALE ANTONIO RUBERTI

SAPIENZA UNIVERSITÀ DI ROMA

Simple HTTP Server

How Socket Works – Stream 2/2



- stream socket = virtual circuit between two processes
- the connection is:
  - Sequenced
  - Reliable
  - Bi-directional (`send()`, `recv()`)
- uniquely identified by the IP addresses and port numbers (remote and local) for:
  - client (`socket()`)
  - server (`socket()`, `bind()`)
- setup by:
  - a server that awaits (`listen()`) for connections (`accept()`)
  - a client that connects to a server (`connect()`)

Simple HTTP Server

Ascoltare le connessioni in arrivo - TCP Sockets in C

```
#include <stdio.h>
#include <string.h>
#include <sys/socket.h>
#include <arpa/inet.h>

int main(void){
    int socket_desc, client_sock, client_size;
    struct sockaddr_in server_addr, client_addr;
    char server_message[2000], client_message[2000];

    // Clean buffers:
    memset(server_message,  \0 , sizeof(server_message));
    memset(client_message,  \0 , sizeof(client_message));
    // Create socket:
    socket_desc = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
    if(socket_desc < 0){/* AF_NET: use IPv4 (no IPv6);
 SOCK_STREAM: TCP (no UDP) */
        printf("Error while creating socket\n");
        return -1;    }
    printf("Socket created successfully\n");

    // Set port and IP:
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(2000);
    server_addr.sin_addr.s_addr = inet_addr("127.0.0.1");

    // Bind to the set port and IP:
    if(bind(socket_desc, (struct sockaddr*)&server_addr,
sizeof(server_addr))<0){
        printf("Couldn t bind to the port\n");
        return -1;    }
    printf("Done with binding\n");
```

```
 // Listen for clients:
    if(listen(socket_desc, 1) < 0){
        printf("Error while listening\n");
        return -1;    }
    printf("\nListening for incoming connections.....\n");

    // Accept an incoming connection:
    client_size = sizeof(client_addr);
    client_sock = accept(socket_desc, (struct sockaddr*)&client_addr,
&client_size);
        if (client_sock < 0){
        printf("Can t accept\n");
        return -1;    }
    printf("Client connected at IP: %s and port: %i\n",
       inet_ntoa(client_addr.sin_addr), ntohs(client_addr.sin_port));

   // Receive client s message:
    if (recv(client_sock, client_message, sizeof(client_message), 0) < 0){
        printf("Couldn t receive\n");
        return -1;    }
    printf("Msg from client: %s\n", client_message);

    // Respond to client:
    strcpy(server_message, "This is the server s message.");
        if (send(client_sock, server_message, strlen(server_message), 0) <
0){
        printf("Can t send\n");
        return -1;    }

    // Closing the socket:
    close(client_sock);
    close(socket_desc);

    return 0;
}
```

DIPARTIMENTO DI INGEGNERIA INFORMATICA
AUTOMATICA E GESTIONALE ANTONIO RUBERTI

SAPIENZA
UNIVERSITÀ DI ROMA

Simple HTTP Server

HTTP/1 – RFC 1945 (1996): Metodi del Protocollo

Il client deve inserire nel messaggio di richiesta quale tipo si servigio vuole richiedere al server.

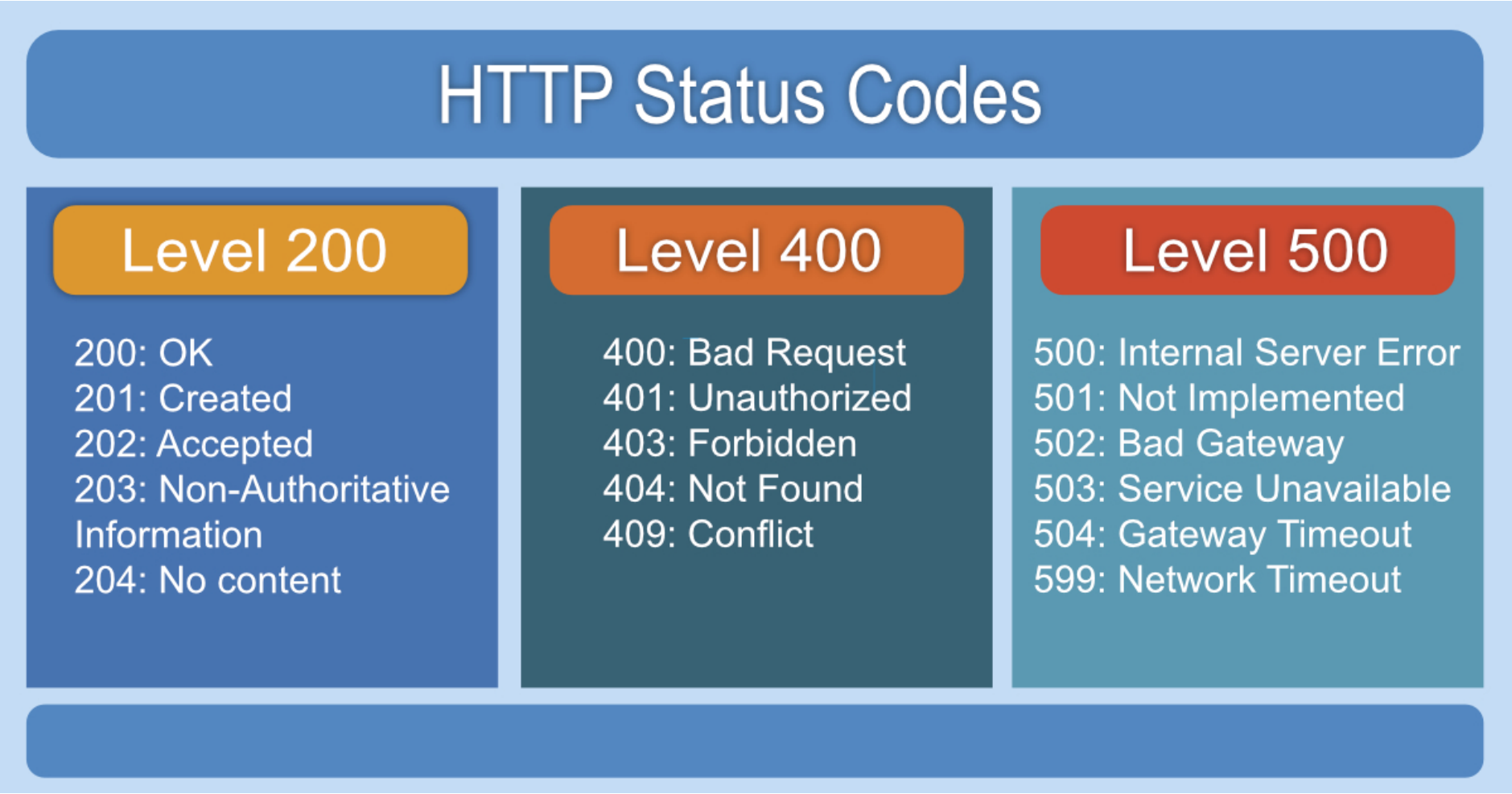
Il simple HTTP server accetta solo il metodo GET



La versione 1.0 del protocollo HTTP prevede solo i metodi: GET, HEAD, POST.
Prevede come addizionali: PUT, DELETE, LINK, UNLINK
Gli altri sono stati aggiunti con HTTP/1.1 (RFC 2616), nel 1999

Simple HTTP Server

HTTP/1 – RFC 1945 (1996): Codici di Stato



Il server deve inserire il codice opportuno nel messaggio di risposta al client.

Simple HTTP Server

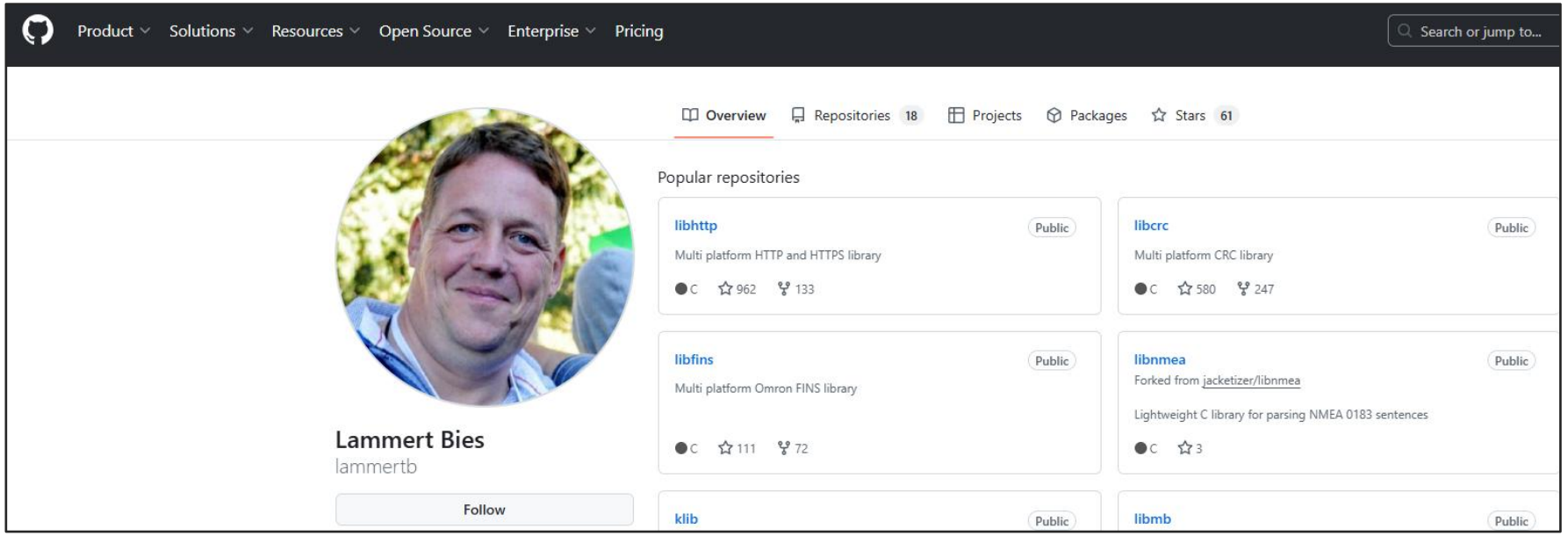
LibHTTP – libreria C

Questa esercitazione è relativa ad un semplice server HTTP, scritto «from scratch», in C. Valido come PoC (Proof of Concept), in modo da poter comprenderne il funzionamento.

Per esigenze operative reali, esistono ausili per non dover riscrivere tutto da capo. Fra questi: LibHTTP: https://www.libhttp.org/

LibHTTP è una libreria con licenza MIT scritta in C che implementa un server HTTP/HTTPS con capacità websocket.

- include anche funzionalità per connessioni client ad altri server.
- è basata sulla famiglia di server HTTP Mongoose(MIT)/Civetweb.
- condivide il codice con questi, sebbene la compatibilità tra chiamate di funzione non sia garantita.
- comprende la documentazione online su come utilizzare la libreria.



Dipartimento di Ingegneria informatica automatica e gestionale Antonio Ruberti

Sapienza Università di Roma

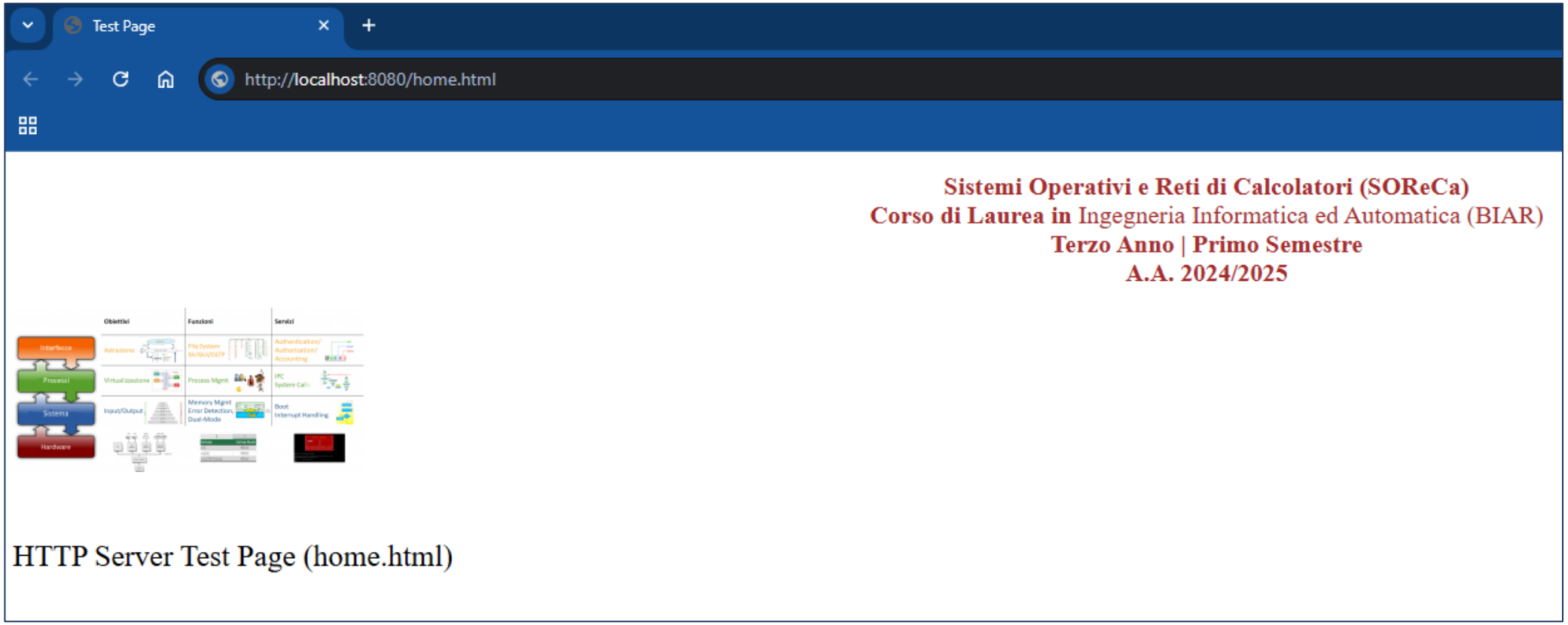
Simple HTTP server

Lab09, es.1 nano HTTP server

- Ascolto: Elaborare Thread-based (Pooling), GET valida, nome file e sua codifica
- Risposta: tipo MIME, Ricerca file e sua Esistenza, Messaggio HTTP

Simple HTTP Server

Lab09, es1 - Server HTTP client: normale browser



Dipartimento di Ingegneria informatica automatica e gestionale Antonio Ruberti

Sapienza Università di Roma

Simple HTTP Server

Lab09, es1 – gestione `stdout`: `snprintf()`

Il server HTTP ha un valore diverso per `stdout`.

`stdout`: per essere visibile al browser → deve essere reindirizzato sul socket di rete, previo passaggio per un buffer di memoria.
Per questo motivo, viene impiegata la funzione `snprintf()`, contenuta in `<stdio.h>`:

```
#include <stdio.h>
int snprintf(char *restrict s, size_t n, const char * format,
...);
```

`s`: puntatore al buffer
`n`: numero massimo di byte che saranno scritti nel buffer
`format`: stringa C che contiene una stringa di formato che segue le stesse specifiche di format in `printf()`.
…: argomenti facoltativi (formati stringa come: “%d”, myint, come per `printf()`).

Nella soluzione, sono usati due buffer:
1. `header`: per costruire l’header della risposta positiva
2. `response`: che contiene il payload (html o img o altro) o la risposta negativa

Simple HTTP Server

Lab09, es1 – gestione `stderr`: `perror()`

Il server HTTP indirizza le informazioni di errore su `stderr`.

`stderr`: per visualizzare anche le informazioni di errore: `errno` settato. Viene impiegata la funzione `perror()`.
Come al solito, è necessario assicurarsi che la funzione `perror()` venga richiamata immediatamente dopo che una funzione della libreria ha restituito un errore; altrimenti, le chiamate successive potrebbero modificare il valore `errno`.

```
#include <stdio.h>
#include <errno.h>
void perror(const char *s);
```

`s`: puntatore alla stringa da stampare (es. "`Could not open data file`")
Stampa
`s` + ": " " " + <messaggio associato al valore in errno> + `\n`

errno	Descrizione
EADDRINUSE	Address already in use
EALREADY	Connection already in progresEs
ENOENT	No such file or directory
EOWNERDEAD	Owner died
EPERM	Operation not permitted

Per una lista complete dei codici di errore disponibili sul Sistema: `$ errno -l`

Dipartimento di Ingegneria informatica automatica e gestionale Antonio Ruberti

Sapienza Università di Roma

Simple HTTP Server

Lab09, es1 - 1 Ascolto:- Elaborare le richieste dei client (thread separati)

Continuare ad ascoltare le richieste degli utenti creando un ciclo continuo (`while(1){[…]}`).

Qui sono accettate le richieste dei client.

Utilizzare le funzioni pthread per invocare le esecuzioni multithread.

Creare una funzione per gestire le richieste dei client: `void *handle_client(void *arg)`

Più processi parallelamente. Quindi più utenti possono utilizzare le risorse del server senza attendere tempi di inattività.

Per ogni nuovo utente, creiamo un nuovo thread per eseguire i suoi processi in isolamento.

Dipartimento di Ingegneria informatica automatica e gestionale Antonio Ruberti

Sapienza Università di Roma

Simple HTTP Server

Lab09, es1 - 2 Ascolto: Verificare se GET valida

Occorre ricevere dati dal lato client, che in genere sono messaggi che richiedono una risorsa specifica dal server.

Quindi, se tale richiesta viene ricevuta correttamente, possiamo controllare il metodo di richiesta per determinare se si tratta di una richiesta GET, PUT, POST, DELETE. Nota che, in questa implementazione, ci concentreremo solo sulle richieste GET.

La generica GET ben-formata è descrivibile dalla RegEx:

```
"^GET /([^ ]*) HTTP/1«
```

È possibile effettuare la comparazione tramite la funzione:

```
#include <regex.h>
int regcomp(regex_t *preg, const char *pattern, int cflags);
```

già implementata. ☺

Dipartimento di Ingegneria informatica automatica e gestionale Antonio Ruberti

Sapienza Università di Roma

Simple HTTP Server

Lab09, es1 - 3 Ascolto: Codifica ed effettivo nome del file

La funzione chiamata `url_decode()` gestisce le decodificazioni delle richieste URL.

Cioé, se l URL richiesto dall utente contiene valori esadecimali, decodifica tali valori in formato UNICODE.

Ad esempio %20 significa spazio e %23 significa #.

Funzione `char *url_decode(const char *src)` già definita. ☺

Dipartimento di Ingegneria informatica
automatica e gestionale Antonio Ruberti

Sapienza
Università di Roma

Simple HTTP Server

Lab09, es1 - 4 Risposta: Estensione del file e tipo MIME

Creare un altra funzione chiamata `get_file_extension()` per leggere ed estrarre il tipo di file.

Ciò è necessario perché nel contesto HTML codificheremo i nostri payload in formato MIME.

Se il nome del file segue FILE_NAME.file_type, allora restituisce file_type come risultato, altrimenti restituisce una stringa vuota "".

```
const char *get_file_extension(const char *file_name) {
    const char *dot = strrchr(file_name, '.');
    if (!dot || dot == file_name) {
        return "";
    } // elimina anche I directory traversal (es. ../../..)
    return dot + 1;
}
const char *get_mime_type(const char *file_ext);
```

già definita. ☺

Dipartimento di Ingegneria informatica automatica e gestionale Antonio Ruberti

Sapienza Università di Roma

Simple HTTP Server

Lab09, es1 - 5 Risposta: Ricerca del file

Poiché il server risponderà con pagine HTML, innanzitutto occorre leggere i file dal disco rigido.

Creare una funzione chiamata `get_file_case_insensitive(const char *file_name)` per reperire il file. già definita. ☺

Questa funzione sostanzialmente prende la directory e il nome del file dai parametri e prova a creare il a directory completa e ad aprire il file effettivo.

Attenzione Maiuscole/minuscole

Dipartimento di Ingegneria informatica automatica e gestionale Antonio Ruberti

Sapienza Università di Roma

Simple HTTP Server

Lab09, es1 - 6 Risposta: Esistenza del file

provare ad aprire la risorsa richiesta (file) e se il file esiste allora possiamo continuare con gli altri passaggi, ma se non esiste allora rispondiamo all utente come
404 Not Found
che significa che la risorsa richiesta non è stata trovata sul server.

Se la risorsa richiesta esiste, allora dobbiamo specificare alcuni metadati associati alla risorsa come Content-Length.

```
int file_fd = open(file_name, O_RDONLY);
    if (file_fd == -1) {
        snprintf(response, BUFFER_SIZE,
                 "HTTP/1.1 404 Not Found\r\n"
                 "Content-Type: text/plain\r\n"
                 "\r\n"
                 "404 Not Found");
        *response_len = strlen(response);
        return;
    }
```

Dipartimento di Ingegneria informatica automatica e gestionale Antonio Ruberti

Sapienza Università di Roma

Simple HTTP Server

Lab09, es1 - 7 Risposta: Creazione messaggio HTTP

Considerando che build_http_response() è una delle funzioni più importanti in questo server web. Perché come suggerisce il nome, questa funzione restituisce le risposte all utente leggendo i file HTML effettivi che risiedono nel disco rigido.

Come primo passaggio, quando l utente richiede una risorsa dalla dimensione del client, dobbiamo restituirla all utente allegando i file di intestazione necessari. Quindi il renderer della dimensione del client, che in genere è un browser web, può facilmente rendere quel contenuto.

Qui stiamo allegando alcuni dei file di intestazione HTTP più importanti, come HTTP/1.1, che determina che utilizzeremo HTTP sulla connessione con lo stato predefinito 200 OK.

```
void build_http_response(const char *file_name,
                         const char *file_ext,
                         char *response,
                         size_t *response_len) {
    // build HTTP header
    const char *mime_type =
get_mime_type(file_ext);
    char *header = (char *)malloc(BUFFER_SIZE *
sizeof(char));
    snprintf(header, BUFFER_SIZE,
             "HTTP/1.1 200 OK\r\n"
             "Content-Type: %s\r\n"
             "\r\n",
             mime_type);
```

Dipartimento di Ingegneria informatica automatica e gestionale Antonio Ruberti

Sapienza Università di Roma

Simple HTTP Server

Lab09, es1 - 8 Risposta: Invio messaggio HTTP

Inviare la risposta sulla connessione HTTP verso il client.

```
send(client_fd, response, response_len, 0);
```

Dipartimento di Ingegneria informatica
automatica e gestionale Antonio Ruberti

Sapienza
Università di Roma

Simple HTTP Server

Lab09, es1

Completare il codice dell'HTTP Server in modalità multi-thread

Sorgenti

- `Makefile`
- Client: <none> usare il browser
- Server: `server.c`
- Pagina HTML: `home.html`
- Immagine nella pagina HTML: `osmatrix3x3_0.png`

Suggerimento: seguire i blocchi di commenti inseriti nel codice

Funzionamento:

- Avviare il server: `./server`
- Aprire nel browser l'URL `http://localhost:8080/home.html`
- Per vedere i thread avviati, da riga di comando:

```
ps -e -T | head -1 ; ps -e -T | grep multithread
```

Dipartimento di Ingegneria informatica
automatica e gestionale Antonio Ruberti

Sapienza
Università di Roma

Autovalutazione

https://forms.gle/LyLAkfooMRc5JYU36